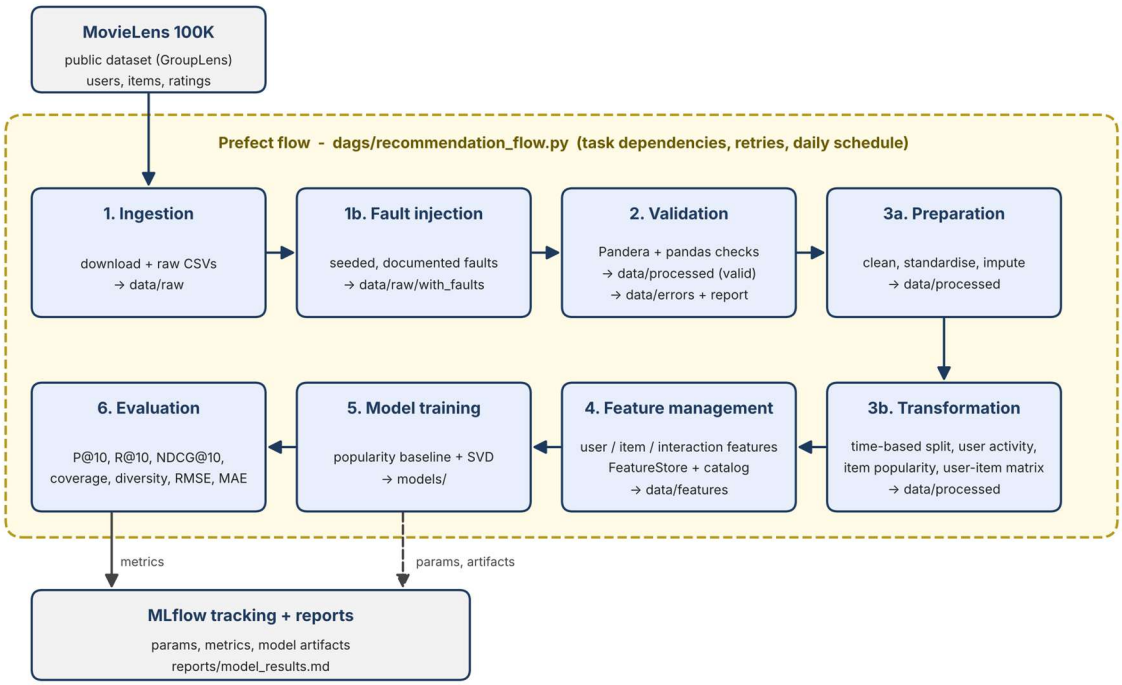
DM4ML Lab Assignment 2: End-to-End Data Management Pipeline for a Recommendation System

Name: Krithika S
Roll No: 2025ag05333 **Date: 13/11/2026**
GitHub: https://github.com/krithi113/recommendation-pipeline

1. Problem and dataset

The goal is to build a clean, modular and reproducible pipeline that takes raw interaction data to a tracked, evaluated recommender, with orchestration. I used MovieLens 100K (943 users, 1,682 movies, 100,000 ratings from 1 to 5, with user demographics and movie genres). More emphasis laid on data management rather than model accuracy.

2. Architecture



Eight Prefect tasks run in sequence, each receiving the previous result so dependencies are explicit. Ingestion has 2 retries. Every stage reads its inputs from files written by the previous one and takes its parameters from one YAML config, so each stage can also be run alone. The flow can be scheduled daily at 02:00.

3. Data validation

MovieLens is clean, so faults are injected with a fixed seed (42) and logged, so the validation report can compare detected counts with injected counts (all matched). Rejected rows are kept in data/errors with their first failing

reason, not silently dropped. **Pandera** handles column checks; pandas handles duplicates and referential integrity. Each bad row gets its first failing reason and is written to data/errors, valid rows go to data/processed.

Table	Input rows	Valid	Rejected	Reasons
users	956	943	13	invalid age 4, missing age 3, invalid gender 3, duplicate 3
items	1,689	1,682	7	missing title 4, duplicate 3
interactions	100,050	99,870	180	duplicate 50, invalid rating 30, invalid timestamp 30, orphan user 20, orphan item 20, missing rating 20, bad dtype 10

Table 1: Records received, accepted and rejected by validation, with the reasons for rejection.

4. Preparation and transformation

Preparation standardises strings, creates age groups, parses release dates, imputes the one missing release year with the median (and flags it), and removes duplicates. Transformation applies a time-based split: the latest 20% of interactions form the test set (cut-off 1998-03-07), giving 79,897 train and 19,973 test rows, with an assertion that train ends before test starts. From train only it builds a user activity summary, item popularity and a sparse user-item matrix (751 x 1,615, sparsity 93.4%). Because of the time split, 192 test users and 67 test items never appear in train (cold-start).

5. Feature management

A feature registry defines every feature with its description, source, type and the model that uses it. A custom offline FeatureStore writes user features (e.g. rating count, average rating), item features (popularity count, average rating, release year) and interaction features, with versioned metadata (v1.0, as-of timestamp). The catalog is generated from the registry and checked against the stored data, so documentation cannot drift. Features use training rows only, which avoids leakage. The full catalog is in reports/feature_catalog.md.

6. Models and tracking

- **Baseline**: popularity recommender that ranks items by training popularity (from the FeatureStore) and predicts ratings with the item average.
- **Personalised model:** PureSVD, a TruncatedSVD with 20 factors (explained variance 0.239) on the mean-centred user-item matrix; the predicted rating is the user average plus the latent score, clipped to [1,5].

Parameters, artifacts and the feature catalog are logged to MLflow (SQLite backend); the evaluation stage resumes the same runs to log metrics, so each model has one complete run.

Tools used: Python and pandas for processing; **Pandera** for validation; Parquet (pyarrow) for storage; scikit-learn (TruncatedSVD) for the personalised model; a custom FeatureStore with an auto-generated catalog; **MLflow** for experiment tracking; **Prefect** for orchestration.

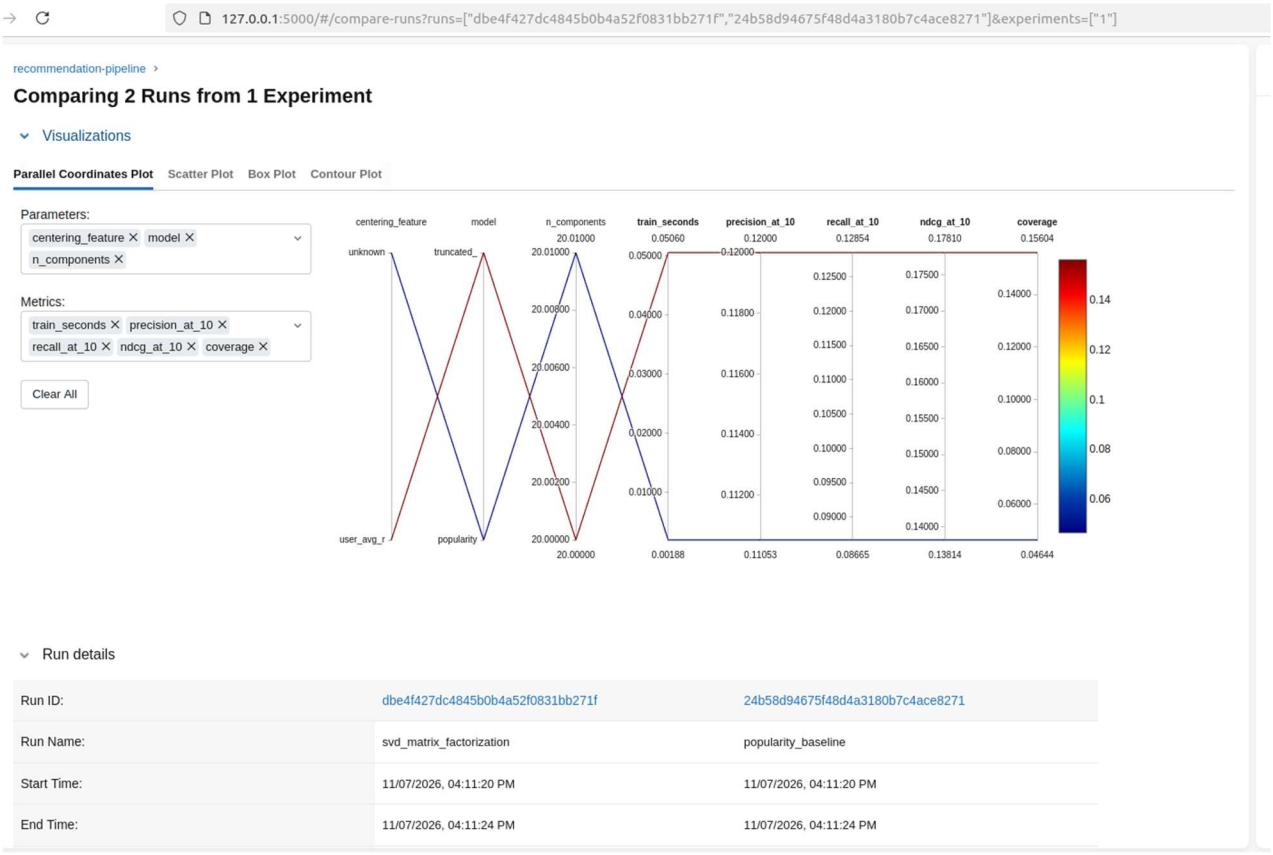


Fig 1: side-by-side comparison (parameters and metrics) for the 2 model runs in MLFlow

7. Evaluation

Protocol: K = 10, warm users and items only, an item is relevant if its test rating is at least 4; metrics average over the 95 users with at least one relevant test item (2,863 warm test rows). Diversity is intra-list genre dissimilarity (1 - cosine); coverage is the share of warm items that appear in recommendations.

Model	P@10	R@10	NDCG@10	Coverage	Diversity	RMSE	MAE
Popularity baseline	0.1105	0.0867	0.1381	0.0464	0.7491	1.0373	0.8336
SVD (20 factors)	0.1200	0.1285	0.1781	0.1560	0.7254	1.1503	0.9251

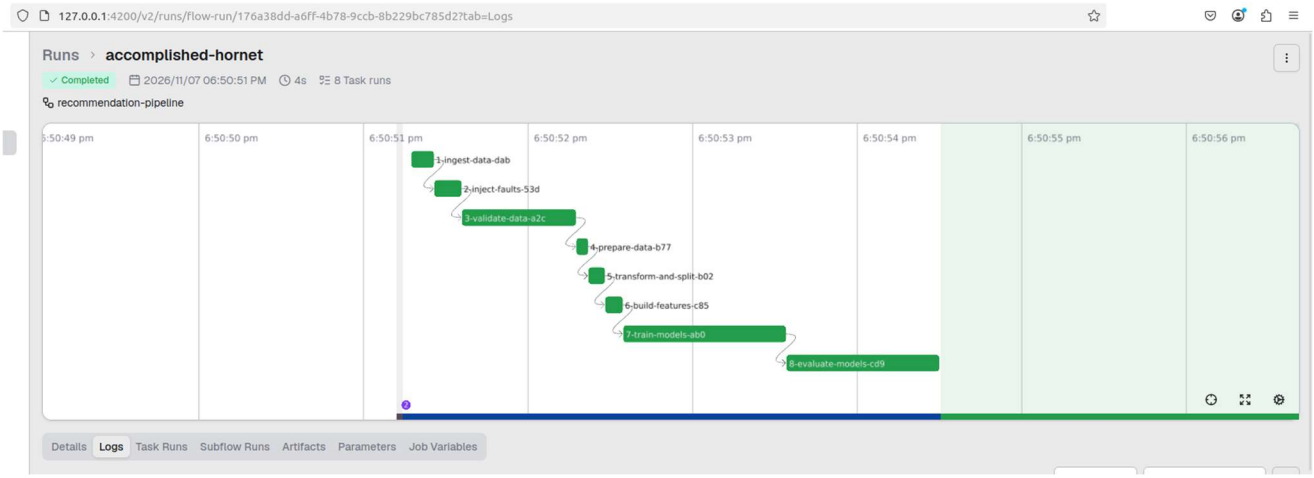
Table 2: Comparison of the popularity baseline and SVD models at K = 10 on the time-based test set (95 evaluated users).

SVD is better on recall, NDCG and catalog coverage (about 3.4 times the baseline), as expected for a personalised model. The precision gap is within the standard error across users, so I do not claim it is significant. The baseline has lower RMSE and MAE and slightly higher diversity. With only 95 evaluated users the comparison is noisy; the point of the assignment is the pipeline, not the ranking of the two models.

8. Orchestration

Roll No: 2025ag05333 | DM4ML Assignment 2

The Prefect flow (dags/recommendation_flow.py) completed successfully end to end; the screenshot of the run is below.



9. Limitations and future work

Cold-start users and items are reported but not served; a content-based or popularity fallback would fix this. The evaluation set is small. The FeatureStore is offline only. Next steps: incremental ingestion, drift monitoring on validated data, and an online feature store.

10. Reproducibility

Python 3.11, versions pinned in requirements.txt, fixed seed (42), one config file. After cloning, "pip install -r requirements.txt" and "python dags/recommendation_flow.py" reproduce every output; the README lists the commands for each stage. The pipeline was run from a fresh clone on both Ubuntu and Windows and produced identical results.